



**Guia #02: Sistema distribuido de registro y consulta de información
con replicación y tolerancia a fallos**

Jose Camilo Pérez Daza

Sebastián Fernando Revelo Meneses

Tomas Alejandro Santiago Reyes

Ingeniería de software, Universidad Manuela Beltrán

Sistemas distribuidos

Prof. Juan José Osorio Tabares

Bogotá D.C.

19 de agosto del año 2026

TABLA DE CONTENIDOS

Introducción	4
Requerimientos	5
Requerimientos funcionales.....	5
Requerimientos no funcionales.....	5
Resumen del sistema.....	7
Repositorio Github.....	11
Instalación y configuración.....	11
Instalación del lenguaje y del run time	11
Instalación de dependencias.....	11
Configuración de red.....	12
Configuración de puertos.....	13
Instalación de la base de datos	13
Ejecución del software.....	14
Comunicación entre servidores.....	15
Evidencias	16
Ambiente de prueba	16
Evidencias de ejecución en ambos servidores	17
Evidencias de comunicación entre servidores	19
Resultados de las pruebas	20
Enlace al video de la ejecución en vivo con el grupo.	22
Conclusiones	23
Referencias.....	24

Declaración final (uso de herramientas IA)	25
--	----

Introducción

En esta guía, realizamos el desarrollo de un cliente y dos servidores de distinto sistema operativo que servirán para registro y consulta de información, cada uno con su instancia de la base de datos, para esto se dividió en dos fases.

En la fase 1 se elaboró el diseño de la propuesta: arquitectura, tecnologías y flujos de comunicación, mientras que este documento es sobre la fase 2, en la que se implementó el sistema diseñado y se recopilaron las evidencias de su funcionamiento.

El sistema está formado por un cliente que usa Ubuntu, un Servidor 1 con Debian, un Servidor 2 con Fedora y una instancia independiente de PostgreSQL por cada servidor, todo adentro de un Docker Compose sobre una red interna con direcciones IP fijas, donde el cliente se comunica con los servidores con API REST, y los servidores tendrán un canal TCP propio, la replicación de registros y la resincronización si llegara a pasar algo.

Requerimientos

Los requerimientos los sacamos de la propuesta que se hizo desde la Fase 1 y de las indicaciones de la guía de la asignatura.

Requerimientos funcionales

Tabla 1

Requerimientos funcionales

Requerimiento funcional	Descripción
RF1	Registrar información desde el cliente mediante un menu en consola.
RF2	Enviar cada registro al servidor correspondiente, y redirigir automáticamente al otro servidor si el primero no responde.
RF3	Consultar la información almacenada desde cualquiera de los dos servidores.
RF4	Mantener la comunicación y sincronía constante entre los dos servidores mediante un heartbeat periódico.
RF5	Replicar cada registro creado en un servidor hacia el otro, sin duplicar datos.
RF6	Conservar en una tabla los registros pendientes de replicar y reintentar su envío mientras el otro servidor no esté disponible.
RF7	Re sincronizar un servidor que se recupera de una caída, hasta que ambos vuelvan a tener los mismos datos.
RF8	Identificar en cada respuesta qué servidor atendió la solicitud.
RF9	Mostrar el estado de conexión de cada servidor y si detecta al otro.
RF10	Registrar en el cliente un historial local en SQLite de las comunicaciones realizadas, sin duplicar los datos de los servidores.

Nota. Elaboración propia.

Requerimientos no funcionales

Tabla 2

Requerimientos no funcionales

Requerimiento no funcional	Concepto	Descripción
RNF1	Tolerancia a fallos	Un servidor cae, otro sigue respondiendo y nada se detiene por la caída de uno
RNF2	Fiabilidad	Nada se pierde, registros se guardan en el base local primero, la réplica va después
RNF3	Rendimiento	Cada servidor accede a su propia base en local. Réplica se dispara al momento, intentando varias veces si falla
RNF4	Idempotencia	Repetir varias veces no duplica nada
RNF5	Portabilidad	Mismo código de servidor, corre igual en Debian y Fedora
RNF6	Simplicidad y mantenibilidad	Código corto, comentado, versionado y una sola implementación, compartida por ambos servidores.
RNF7	Aislamiento	Red interna que no abre puertos de más hacia el anfitrión

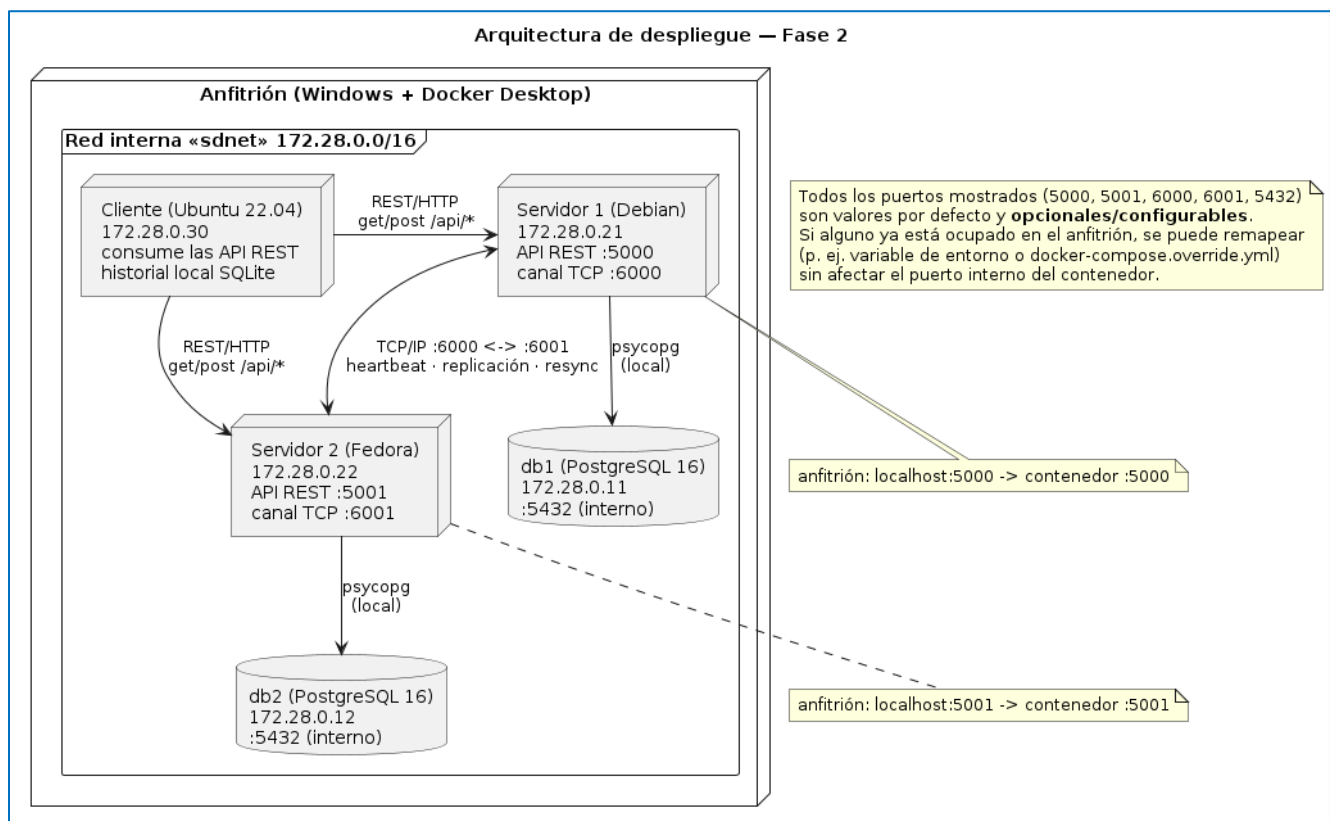
Nota. Elaboración propia.

Resumen del sistema

Para el despliegue tenemos cinco contenedores con una red interna sdnet (172.28.0.0/16), donde está el cliente (Ubuntu 22.04), el Servidor 1 (Debian) con su base db1, y el Servidor 2 (Fedora) con su base db2. Cada servidor corre su API REST en Flask y su propio PostgreSQL. Cliente consume ambas API, guarda historial local en SQLite. Ahora la Figura 1 muestra la arquitectura de despliegue.

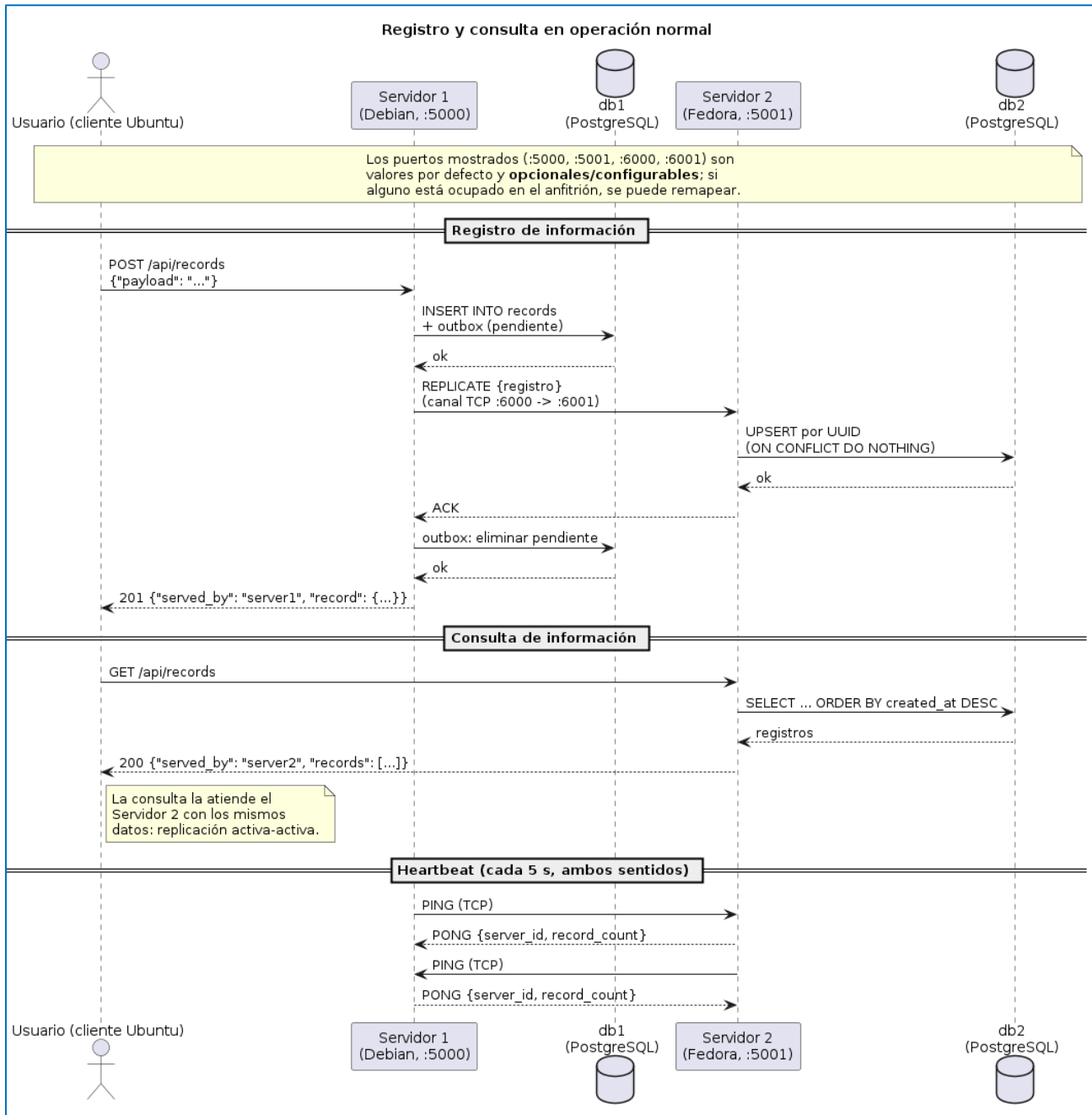
Figura 1

Arquitectura de despliegue del sistema distribuido



Nota. Elaboración propia con PlantUML, actualizado con las nuevas adaptaciones.

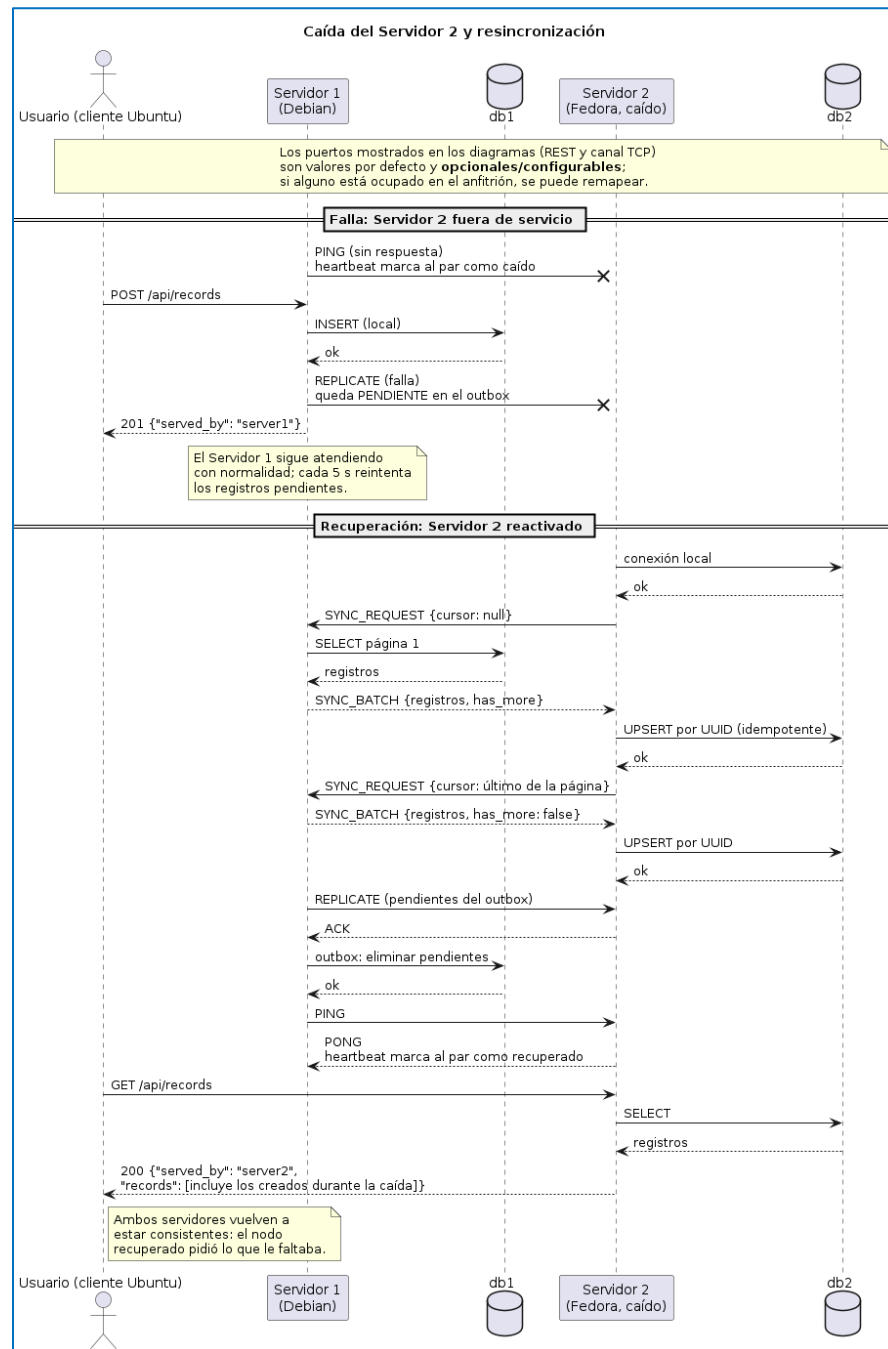
La comunicación cliente-servidor usa REST sobre HTTP, entre servidores existe además un canal TCP propio (puertos 6000 y 6001, solo en la red interna) con mensajes JSON de una línea y tres funciones: un heartbeat cada 5 segundos (PING/PONG) que permite a cada nodo conocer el estado del par; la replicación por empuje, en la que cada registro local se envía de inmediato al par (REPLICATE/ACK) y, si el par no responde, queda pendiente en una tabla outbox y se reintenta cada 5 segundos; y la resincronización por arrastre (SYNC_REQUEST/SYNC_BATCH), que se ejecuta al arrancar, al detectar la recuperación del par y cada 60 segundos como red de seguridad. Todas las inserciones son idempotentes por UUID, de modo que repetir un mensaje nunca duplica datos. La Figura 2 ilustra el flujo normal de registro y consulta, y la Figura 3 el escenario de caída y recuperación.

Figura 2*Registro y consulta en operación normal*

Nota. Elaboración propia con PlantUML.

Figura 3

Ejemplo caído del Servidor 2 y como se resincroniza



Nota. Elaboración propia con PlantUML.

Repositorio Github

A continuación, adjuntamos el repositorio de nuestro código:

<https://github.com/sebas-archive/servidores-y-bases-de-datos-distribuida>

Instalación y configuración

Ahora se verá las siete cosas solicitadas en las instrucciones de la Fase 2: El detalle completo, junto con las variables de configuración y la solución de problemas, está en el manual de instalación y configuración del repositorio.

Instalación del lenguaje y del run time

El lenguaje del proyecto es Python 3, instalado dentro de cada contenedor con el gestor de paquetes de su distribución: apt en el cliente (Ubuntu 22.04) y en el Servidor 1 (Debian), y dnf en el Servidor 2 (Fedora). No se instala ningún lenguaje ni run time en el anfitrión; la instalación ocurre automáticamente durante la construcción de las imágenes con Docker Compose.

Instalación de dependencias

Las dependencias están declaradas en archivos requirements.txt y se instalan con pip durante el build de cada imagen: Flask y psycopg (driver de PostgreSQL) en los servidores, y requests en el cliente. En Debian se usa el flag --break-system-packages porque su Python de sistema está marcado como externally managed (PEP 668).

Configuración de red

Docker Compose crea una red bridge interna llamada sdnet con el rango 172.28.0.0/16 y direcciones IP fijas para cada servicio (Tabla 1). Los servicios se alcanzan entre sí por sus nombres DNS internos, que son más legibles y estables que las IP.

Tabla 3

Direcciones IP y nombres DNS de la red interna sdnet

Servicio	IP en sdnet	Nombre DNS interno
db1 (PostgreSQL de server1)	172.28.0.11	db1
server1 (Debian)	172.28.0.21	server1
db2 (PostgreSQL de server2)	172.28.0.12	db2
server2 (Fedora)	172.28.0.22	server2
client (Ubuntu)	172.28.0.30	client

Nota. Elaboración propia con base en docker-compose.yml del repositorio.

Configuración de puertos

Cada servidor usa dos puertos internos: el de la API REST y el del canal TCP de sincronía. Solo los puertos de la API se publican en el anfitrión, para probar con el navegador o con curl; PostgreSQL y el canal TCP permanecen dentro de la red interna:

Tabla 4

Puertos de cada servicio

Servicio	Puerto interno	Publicado en el anfitrión	Uso
server1	5000 (Flask)	localhost:5000	API REST del Servidor 1
server1	6000 (TCP propio)		Heartbeat, replicación y resincronización
server2	5001 (Flask)	localhost:5001	API REST del Servidor 2
server2	6001 (TCP propio)		Heartbeat, replicación y resincronización
db1 / db2	5432 (PostgreSQL)		Base local de cada servidor
client			Solo consumidor

Nota. Elaboración propia con base en docker-compose.yml del repositorio.

Instalación de la base de datos

Cada servidor administra su propia instancia de PostgreSQL 16 mediante la imagen oficial, con la base records, el usuario sduser y volúmenes nombrados (db1_data y db2_data) para persistir los datos. El esquema se crea automáticamente al arrancar cada servidor (sentencias CREATE TABLE IF NOT EXISTS) con dos tablas: records (registros, clave UUID) y outbox (registros locales pendientes de replicar). El cliente, por su parte, guarda su historial en una base SQLite dentro de un volumen propio.

Ejecución del software

El sistema se levanta con un único comando de Docker Compose, seguido de la espera de los healthchecks; el cliente se ejecuta con un menú interactivo o con subcomandos:

```
docker compose up -d --build db1 db2 server1 server2
```

```
docker compose run --rm --no-deps client          # menú interactivo
```

```
docker compose run --rm --no-deps -T client python3 client.py create --payload "Hola mundo" --  
server 1
```

```
docker compose run --rm --no-deps -T client python3 client.py list --server 2
```

```
docker compose run --rm --no-deps -T client python3 client.py status
```

Comunicación entre servidores

Los servidores se comunican por un canal TCP propio con mensajes JSON de una línea (puertos 6000 y 6001). El protocolo tiene cuatro tipos de mensaje: PING/PONG para el heartbeat cada 5 segundos; REPLICATE/ACK para empujar un registro al par; SYNC_REQUEST/SYNC_BATCH para pedir los registros del par por páginas de 100, ordenadas por fecha e identificador; y ERROR para respuestas no reconocidas. La actividad del canal queda registrada en los logs con el prefijo [sync] y puede observarse con docker compose logs. La configuración principal de cada servidor se resume en:

Tabla 5

Variables de entorno principales de los servidores

Variable	server1	server2	Significado
SERVER_ID	server1	server2	Identidad del nodo (aparece en served_by)
PORT	5000	5001	Puerto de la API REST
SYNC_PORT	6000	6001	Puerto del canal TCP de sincronía
PEER_HOST	server2	server1	Nombre DNS del servidor par
HEARTBEAT_SECONDS / REPLICATE_SECONDS	5 / 5	5 / 5	Intervalos de heartbeat y reintentos
SYNC_EVERY_SECONDS	60	60	Resincronización periódica de seguridad

Nota. Elaboración propia con base en docker-compose.yml del repositorio.

Evidencias

Esta sección reúne los entregables 5, 6 y 7 de las instrucciones de la Fase 2: evidencias de ejecución en ambos servidores, evidencias de comunicación entre servidores y resultados de las pruebas. Las capturas fueron tomadas por el equipo durante la ejecución del sistema con Windows y Docker Desktop.

Ambiente de prueba

Tabla 6

Datos del ambiente de prueba

Campo	Valor
Fecha de ejecución	18 de agosto del 2026
Equipo	Jose Camilo Pérez Daza Sebastián Fernando Revelo Meneses Tomas Alejandro Santiago Reyes
Sistema operativo anfitrión	Windows
Docker / Docker Compose	29.7.2
Estado inicial	Despliegue limpio antes de comenzar

Nota. Elaboración propia.

Evidencias de ejecución en ambos servidores

La Figura 4 muestra los cuatro contenedores en estado healthy tras el arranque. La Figura 5 confirma que cada servidor ejecuta su distribución correspondiente: Debian GNU/Linux en el Servidor 1 y Fedora Linux en el Servidor 2 (archivo `/etc/os-release`).

Figura 4

Estado de los contenedores después del arranque (docker compose ps)

☐		Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk read/write	Network I/O	Actions
☐	●	client-run-c5e5t	eb6a75bd94fc	servidores:		0%	20.47MB / 15.34	0.13%	23.4MB / 381KB	8.55KB / 6.89KB	
☐	> ○	guia-003	-	-	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	
☐	▼ ●	servidores-y-ba	-	-	-	4.84%	181.88MB / 61.3	1.15%	87.9MB / 108.03	290.1KB / 287.3	
☐	●	sd2-db2	806bdc6457ec	library/pos		3.79%	55.58MB / 15.34	0.35%	42.9MB / 54.1ME	87.3KB / 96.8KB	
☐	●	sd2-db1	05a7f177861c	library/pos		0.66%	56.73MB / 15.34	0.36%	43.2MB / 53.9ME	61KB / 65.7KB	
☐	●	sd2-server1	dda5b66013d4	servidores: 5000:5000 ↗		0.02%	32.72MB / 15.34	0.21%	0B / 0B	26.8KB / 23.8KB	
☐	●	sd2-server2	721f21aca900	servidores: 5001:5001 ↗		0.37%	36.85MB / 15.34	0.23%	1.2MB / 28.7KB	115KB / 101KB	

Nota. Captura de pantalla del equipo. Elaboración propia.

Figura 5

Distribución de cada servidor (/etc/os-release)

```
server > Dockerfile.debian
1  # Servidor 1 - Debian GNU/Linux
2  #
3  # Imagen base y gestor de paquetes propios de Debian; el código de la
4  # aplicación es el mismo que usa el Servidor 2 (Fedora).
5  FROM debian:stable
6
7  RUN apt-get update \
8  && apt-get install -y --no-install-recommends python3 python3-pip \
9  && rm -rf /var/lib/apt/lists/*
10
11 WORKDIR /app
12
13 COPY requirements.txt ./
14 # Debian marca el Python del sistema como "externally managed" (PEP 668),
15 # por eso se instala con --break-system-packages dentro del contenedor.
16 RUN pip3 install --no-cache-dir --break-system-packages -r requirements.txt
17
18 COPY . .
19
20 EXPOSE 5000 6000
21
22 CMD ["python3", "app.py"]
23
```

```
server > Dockerfile.fedora
1  # Servidor 2 - Fedora Linux
2  #
3  # Imagen base y gestor de paquetes propios de Fedora; el código de la
4  # aplicación es el mismo que usa el Servidor 1 (Debian).
5  FROM fedora:latest
6
7  RUN dnf install -y python3 python3-pip \
8  && dnf clean all
9
10 WORKDIR /app
11
12 COPY requirements.txt ./
13 RUN pip3 install --no-cache-dir --break-system-packages -r requirements.txt
14
15 COPY . .
16
17 EXPOSE 5001 6001
18
19 CMD ["python3", "app.py"]
20
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Evidencias de comunicación entre servidores

La Figura 6 muestra la creación de un registro a través del Servidor 1 y del Servidor 2, lo que evidencia la replicación entre las dos bases.

Figura 6

Registro creado a través de ambos servidores.

```
Opción: 2
2Registros almacenados – atendido por server2 (3 mostrados):
[2026-08-18T21:22:47.205971+00:00] (server2) 91557c45 – prueba3
[2026-08-18T21:22:19.000352+00:00] (server1) 45a3a000 – prueba2
[2026-08-18T21:22:11.477841+00:00] (server1) 31d47ec5 – prueba1
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Figura 7

Comprobación directa del canal TCP (PING/PONG)

```
sd2-server2 | 21:23:43 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server1 | 21:24:40 [sync] par recuperado; lanzando resincronizaci|n
sd2-server1 | 21:24:40 [sync] resincronizaci|n completada: 1 registro(s) nuevos
sd2-server2 | 21:23:52 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server1 | 21:24:43 [sync] resincronizaci|n completada: 0 registro(s) nuevos
sd2-server1 | 21:24:43 [sync] petici|n de resincronizaci|n: 3 registro(s), cursor=None
sd2-server2 | 21:24:01 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server2 | 21:24:14 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server2 | 21:24:23 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server2 | 21:24:36 [sync] par inalcanzable; el registro 91557c45 queda pendiente
sd2-server2 | 21:24:40 [sync] petici|n de resincronizaci|n: 3 registro(s), cursor=None
sd2-server1 | 21:25:43 [sync] petici|n de resincronizaci|n: 3 registro(s), cursor=None
sd2-server2 | 21:24:41 [sync] registro 91557c45 replicado al par
sd2-server1 | 21:25:45 [sync] resincronizaci|n completada: 0 registro(s) nuevos
sd2-server2 | 21:24:41 [sync] replicaci|n: 1 registro(s) pendiente(s) confirmados
sd2-server2 | 21:24:43 [sync] petici|n de resincronizaci|n: 3 registro(s), cursor=None
sd2-server2 | 21:24:43 [sync] par recuperado; lanzando resincronizaci|n
sd2-server2 | 21:24:43 [sync] resincronizaci|n completada: 0 registro(s) nuevos
sd2-server2 | 21:25:43 [sync] resincronizaci|n completada: 0 registro(s) nuevos
sd2-server2 | 21:25:45 [sync] petici|n de resincronizaci|n: 3 registro(s), cursor=None
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Resultados de las pruebas

Se ejecutaron dos escenarios, automatizados con los scripts de la carpeta tests del repositorio y verificados paso a paso de forma manual.

Escenario 1, operación normal: se levantó el sistema completo; se creó un registro a través del Servidor 1; se consultó desde el Servidor 2 (el registro ya estaba replicado); y se verificó el estado de ambos servidores y su visión del par.

Figura 8

Estado de ambos servidores en operación normal

```
==== Cliente – Sistema distribuido de registros (Fase 2) ====
1. Crear registro
2. Consultar registros
3. Estado de los servidores
4. Ver historial local (SQLite)
0. Salir

Opción: 3
Servidor 1 (Debian) [server1]: en línea, 3 registro(s), ve al par en línea
Servidor 2 (Fedora) [server2]: en línea, 3 registro(s), ve al par en línea
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Escenario 2, caída y recuperación: se detuvo el Servidor 2 y el heartbeat del Servidor 1 detectó la caída (Figura 9); se crearon dos registros con el Servidor 2 caído, uno de ellos pidiendo explícitamente el Servidor 2 para evidenciar la conmutación automática del cliente (Figura 10); se reactivó el Servidor 2 y, tras el heartbeat y la resincronización, este recuperó los registros creados durante su caída (Figura 11); por último se verificó el estado final y el historial SQLite del cliente (Figura 12).

Figura 9

Estado con el Servidor 2 detenido (detección por heartbeat)

☒	●	sd2-server1	b852915ed3b3	servidores: 5000:5000	0.3%	33.6MB / 15.34G	0.21%	0B / 28.7KB	35.7KB / 30.2KB
☐	○	sd2-server2	160ee97c0861	servidores: 5001:5001	0%	0B / 0B	0%	0B / 0B	0B / 0B

```
Opción: 3
Servidor 1 (Debian) [server1]: en línea, 0 registro(s), ve al par caído
Servidor 2 (Fedora): NO RESPONDE
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Figura 10

Registros creados durante la caída, con conmutación del cliente

```
Opción: 1
Contenido del registro: PruebaServercaido
ERROR: servidor 2 inalcanzable

==== Cliente - Sistema distribuido de registros (Fase 2) ====
1. Crear registro
2. Consultar registros
3. Estado de los servidores
4. Ver historial local (SQLite)
0. Salir

Opción: 2
Registros almacenados - atendido por server1 (1 mostrados):
[2026-08-18T21:31:19.699793+00:00] (server1) 49e1cb44 - PruebaServercaido
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Figura 11

Servidor 2 resincronizado con los registros creados durante su caída

```
Opción: 3
Servidor 1 (Debian) : NO RESPONDE
Servidor 2 (Fedora) [server2]: en línea, 1 registro(s), ve al par en línea

==== Cliente - Sistema distribuido de registros (Fase 2) ====
1. Crear registro
2. Consultar registros
3. Estado de los servidores
4. Ver historial local (SQLite)
0. Salir

Opción: 2
Registros almacenados - atendido por server2 (1 mostrados):
[2026-08-18T21:31:19.699793+00:00] (server1) 49e1cb44 - PruebaServercaído
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Figura 12

Estado final de los servidores e historial SQLite del cliente

```
==== Cliente - Sistema distribuido de registros (Fase 2) ====
1. Crear registro
2. Consultar registros
3. Estado de los servidores
4. Ver historial local (SQLite)
0. Salir

Opción: 4
CUÁNDO (UTC)      ACCIÓN  OBJETIVO  ATENDIÓ  RESULTADO  DETALLE
2026-08-18T21:33:37+00:00 list    auto     server2  ok        1 registro(s)
2026-08-18T21:33:23+00:00 status  2       server2  ok
2026-08-18T21:33:23+00:00 status  1       -        error    servidor 1 inalcanzable
2026-08-18T21:31:31+00:00 list    auto     server1  ok        1 registro(s)
2026-08-18T21:31:26+00:00 create auto     -        error    servidor 2 inalcanzable
2026-08-18T21:30:54+00:00 status  2       -        error    servidor 2 inalcanzable
2026-08-18T21:30:50+00:00 status  1       server1  ok
```

Nota. Captura de pantalla del equipo. Elaboración propia.

Enlace al video de la ejecución en vivo con el grupo.

A continuación, adjuntamos el enlace a nuestro de video de YouTube para ver la ejecución de cómo funciona el sistema:

<https://youtu.be/wZPjzQO7w9Q>

Conclusiones

En definitiva, se realizó el sistema que se planteaba en la fase 1, donde lo probamos corriendo el mismo código en dos distribuciones distintas, Debian y Fedora, estos dos tienen replicación activa entre db1 y db2, la cual estuvo bien, ya que un registro creado en un servidor aparecía en el momento en el otro, y cada respuesta indicaba qué servidor la había atendido.

Para probar la tolerancia a fallos tumbamos el Servidor 2 a propósito, el Servidor 1 notó la caída por el heartbeat y siguió atendiendo al cliente sin interrupción, mientras los registros creados durante la caída quedaban guardados como pendientes y cuando el nodo volvió, el sistema se sincronizó solo, gracias a los reintentos que hace cada vez y a la resincronización, sin que tuviéramos que hacer algo más.

De todo esto al final nos quedan tres cosas claras, primero, que usar contenedores hace que correr el mismo código en distribuciones distintas y levantar todo el sistema con un solo comando sea casi trivial. Segundo, que no hacía falta nada sofisticado para el protocolo de comunicación, un protocolo propio sobre TCP fue suficiente para manejar heartbeat, replicación y resincronización. Tercero, que las pruebas de caída dejaron claro que registrar lo pendiente y reintentar vale mucho más que simplemente fallar y listo.

Referencias

- Anthropic. (2026). Claude [Modelo de lenguaje de gran tamaño]. <https://claude.ai>
- Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. (2011). Distributed systems: Concepts and design (5.^a ed.). Addison-Wesley.
- Docker Inc. (s. f.). Docker Compose overview. <https://docs.docker.com/compose/>
- Osorio Tabares, J. J. (2026). Guía #02: Comunicación en sistemas distribuidos [Material del curso]. Sistemas Distribuidos, Universidad Manuela Beltrán.
- Pallets Projects. (s. f.). Flask documentation. <https://flask.palletsprojects.com/>
- PlantUML. (s. f.). PlantUML language reference guide. <https://plantuml.com/>
- PostgreSQL Global Development Group. (s. f.). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
- Python Software Foundation. (s. f.). The Python standard library. <https://docs.python.org/3/library/>

Declaración final (uso de herramientas IA)

Herramientas de inteligencia artificial fueron empleadas únicamente como apoyo durante el desarrollo de la Fase 2, y que el código, las configuraciones, las pruebas y las evidencias presentadas en este documento fueron ejecutadas y verificadas por los integrantes del equipo.